# ResolveDesk

01 Create, assign, prioritize and resolve support tickets.

02 React interface, FastAPI services and persistent PostgreSQL storage.

03 Built for the DevSecOps assignment. Temporary AWS resources are removed after evidence capture.

# A small workflow, implemented end to end

01 Requester email, subject, description, priority and assignee stay with each ticket.

02 The workflow is OPEN -> IN_PROGRESS -> RESOLVED; teams can reopen tickets.

03 Search and filters help triage; summary counts expose unresolved urgent work.

04 Assumption: an internal classroom demonstration with synthetic data and no authentication.

# Separate responsibilities

01 React + Vite: ticket workspace, forms and accessible feedback.

02 FastAPI: HTTP routes; service functions: workflow operations; Pydantic: validation.

03 SQLAlchemy + PostgreSQL: storage. Alembic: explicit schema migrations.

04 Docker Compose: local integration. Helm + Kubernetes: repeatable deployment.

# One application, two delivery paths

**Browser** -> **React / Nginx** -> **FastAPI** -> **PostgreSQL**

GitHub Actions -> GHCR -> ArgoCD / Helm -> EKS

Prometheus <- /metrics Grafana <- Prometheus

01 Local: Compose connects the same three application services.

02 AWS: Terraform creates the VPC and EKS; Helm defines the workload.

# Test before publication

01 GitHub Actions runs backend tests, frontend tests and a production frontend build.

02 Build each image once and tag it with the source commit SHA.

03 Trivy blocks HIGH and CRITICAL findings and preserves scan reports.

04 Deploy scanned images to temporary Kind, smoke-test them, then publish to GHCR.

# Make infrastructure reproducible

01 VPC spans two availability zones, with public and private subnets.

02 EKS worker nodes use private subnets; a NAT gateway provides outbound access.

03 The Kubernetes API public endpoint is restricted to explicit operator CIDRs.

04 EBS CSI receives a dedicated IAM role for persistent volumes.

05 Capture plan/apply and AWS proof, then destroy temporary billable resources.

# Deploy and recover predictably

01 Frontend and backend Deployments use probes, resource requests and limits.

02 A database StatefulSet uses persistent storage; credentials come from a Secret.

03 A migration Job applies Alembic before the application is considered ready.

04 HPA adjusts backend replicas; an Ingress can provide a single application endpoint.

05 Runtime evidence must show the actual pods, services, storage and health state.

# Git records the desired release

01 ArgoCD reads the Helm chart and release values from Git.

02 Commit-SHA image tags connect a deployed release to its tested source.

03 A values change updates the desired state; ArgoCD reconciles the cluster.

04 Show Synced and Healthy states, then demonstrate a controlled release update.

05 A configured Application alone is not proof of a successful synchronization.

# Observe behavior, not just containers

01 /health reports process liveness; /ready checks access to the migrated database.

02 /metrics exposes HTTP request counts, duration and status metrics.

03 A ServiceMonitor connects the API to Prometheus; verify its target is UP.

04 Grafana displays request rate, p95 latency and server errors.

05 Generate traffic and allow scrape intervals before recording the dashboard.

# Evidence and lessons

01 Verified: 26 isolated backend tests, including validation, CRUD, readiness and migration waiting.

02 Verified: Alembic migration round-trip and real HTTP CRUD against the local PostgreSQL deployment.

03 Runtime evidence: see demo/evidence and the final walkthrough for the recorded deployment state.

04 Keep test results, deployed state and teardown proof separate; none implies the others.

05 The demo has no authentication and uses a single PostgreSQL instance; production needs access control and a recovery plan.